UNIVERSAL CODE SCANNER

05 · Referencia técnica

Documentación de sistema

Versión analizada: **1.1.0+2**
Commit: **6d5e0b7**
Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de **docs/system-documentation/** con **tool/build_system_documentation_pdf.py**. El Markdown es la fuente: no editar este PDF.

Contenido

1. Constantes públicas
2. Enumeraciones
3. Cifrado — `PayloadCipher`
4. Rotación — `DataMaintenanceService.rotateEncryptionKey()`
5. Análisis de riesgo — `ScanSecurityAnalyzer`
6. Interpretación — `ContentInterpreter.parse(String rawValue)`
7. Registro de parsers — `ContentParserRegistry`
8. Historial — `HistoryRepository`
9. Importación — `ImportService`
10. Exportación — `ExportService`
11. Confirmación de lectura — `ScanFeedback`
12. Modelos — firmas clave
13. Claves de `SharedPreferences`
14. Almacenes Sembast
15. Códigos de error e identificadores internos
16. Límites operativos

Catálogo de consulta. Para cada elemento relevante: firma, propósito, parámetros, retorno, excepciones, efectos secundarios, quién lo llama, a quién llama y **riesgo al modificarlo**.

La explicación narrativa de los mismos módulos está en 06-deep-code-explanation.md.

Constantes públicas

Constante	Valor	Archivo	Consecuencia de cambiarla
`appVersion`	`'1.1.0'`	`core/app_info.dart`	`validate_structure.py` falla si no coincide con `pubspec.yaml`
`appName`	`'Universal Code Scanner'`	`core/app_info.dart`	Solo texto de interfaz
`SchemaMigrator.currentVersion`	`4`	`core/database/schema_migrator.dart`	Bajarla deja bases existentes por delante del código
`PayloadCipher.currentVersion`	`2`	`core/security/payload_cipher.dart`	**Crítico.** Un sobre con versión mayor se rechaza con `FormatException`
`PayloadCipher.currentAlgorithm`	`'AES-256-GCM'`	idem	**Crítico.** Un sobre con otro algoritmo se rechaza
`PayloadCipher.currentKeyId`	`'v2'`	idem	**Crítico.** Cambiarla busca una llave que no existe
`PayloadCipher.legacyKeyId`	`'v2'`	idem	Idéntica a la anterior; ver la nota más abajo
`SecureStorageKeyProvider._prefix`	`'ucs_database_key_'`	idem	**Crítico.** Cambiarlo pierde el acceso a las llaves guardadas
`AppDiagnostics.maxEntries`	`100`	`core/diagnostics/app_diagnostics.dart`	Solo memoria
`HistoryRepository.maxItems`	`5000`	`services/history_repository.dart`	Recorte del historial
`ImportService.currentSchemaVersion`	`2`	`services/import_service.dart`	Un respaldo con versión mayor se rechaza
`ImportService.maxHistoryRecords`	`5000`	idem	Límite de registros por respaldo
`ImportService.maxImportBytes`	`25 * 1024 * 1024`	idem	25 MB
`ImportService.maxJsonDepth`	`32`	idem	Defensa contra JSON anidado
`ImportService.maxJsonNodes`	`100000`	idem	Defensa contra JSON masivo
`ImportService.maxInventoryItems`	`10000`	idem	Límite de productos por sesión

`AssetScanTonePlayer.assetPath`	`'sounds/scan_success.wav'`	`services/scan_feedback.dart`	Ruta relativa a `assets/`, tal como la resuelve `AssetSource`
`HandheldFrame.maxWidth`	`560`	`app.dart`	Umbral de centrado en pantallas anchas

Nota sobre `legacyKeyId`: vale `'v2'`, exactamente igual que `currentKeyId`. Un sobre «heredado» —el que no tiene campo `version`— se descifra por tanto con la misma llave que uno actual. Es correcto para este repositorio (el formato inicial usaba esa misma llave) pero las dos constantes son indistinguibles y cualquier cambio futuro de `currentKeyId` **rompería silenciosamente la lectura de los sobres heredados** si no se cambia también `legacyKeyId`. R-13.

Enumeraciones

Enum	Valores	Archivo
`RiskLevel`	`low`, `caution`, `high`	`core/security/scan_security_analyzer.dart`
`ContentKind`	`url`, `wifi`, `contact`, `event`, `email`, `phone`, `sms`, `geo`, `otp`, `gs1`, `isbn`, `product`, `payment`, `crypto`, `identity`, `binary`, `text` (17)	`models/parsed_content.dart`
`AppLanguage`	`system`, `esCl`, `es`, `en`	`models/app_settings.dart`
`ScanPhase`	`starting`, `scanning`, `paused`, `unavailable`	`features/scanner/widgets/scan_status_bar.dart`
`RecoveryEntityType`	`history`, `inventory`, `migration`, `database`, `startup`	`core/recovery/recovery_issue.dart`
`RecoveryIssueState`	`unresolved`, `recovered`, `deleted`	idem
`ImportStrategy`	`merge`, `skipDuplicates`, `replace`	`services/import_service.dart`
`_PayloadType` (privado)	`text`, `url`, `wifi`, `contact`, `event`, `email`, `phone`, `sms`, `geo`	`features/generator/generator_screen.dart`
`_CodeFormat` (privado)	`qr`, `dataMatrix`, `aztec`, `pdf417`, `code128`, `code39`, `ean13`, `ean8`, `upcA`, `gs128`	idem

`ContentKind`, `RiskLevel`, `RecoveryEntityType` y `RecoveryIssueState` se serializan **por nombre** (`.name` / `.byName`). Renombrar un valor rompe la lectura de los datos ya guardados y de los respaldos exportados.

Cifrado — `PayloadCipher`

`Future<String> encryptJson(Map<String, dynamic> value, {String? keyId})`

Aspecto	Detalle
Propósito	Cifrar un mapa JSON y devolver el sobre serializado
Parámetros	`value`: el objeto a cifrar. `keyId`: llave alternativa; por defecto la activa

Retorno	JSON con `version`, `algorithm`, `keyId`, `createdAt` y `payload{nonce, cipherText, mac}`, todos en Base64
Excepciones	`StateError('encryption_key_missing:<keyId>')` si ese identificador está marcado como ausente
Efectos secundarios	**Crea la llave si no existe** (`readOrCreate`) y la escribe en el almacén seguro
Lo llama	`HistoryRepository` (4 sitios), `InventoryRepository.save`, `DataMaintenanceService.rotateEncryptionKey`, `upgradeEnvelope`
Riesgo al modificar	**Crítico.** Cualquier cambio de forma del sobre invalida los datos existentes

La comprobación de `_missingKeyIds` **al cifrar** es la mitad menos evidente y más importante del diseño: sin ella, un descifrado fallido seguido de una escritura crearía una llave nueva bajo el mismo identificador y dejaría los datos antiguos ilegibles para siempre.

`Future<Map<String, dynamic>> decryptJson(String encoded)`

Aspecto	Detalle
Retorno	El mapa original
Excepciones	`FormatException` si la versión está fuera de `[1, currentVersion]` o el algoritmo no es `AES-256-GCM`; `StateError('encryption_key_missing:<keyId>')` si la llave no está; `SecretBoxAuthenticationError` de `cryptography` si el MAC no valida
Efectos secundarios	Añade o quita el `keyId` de `_missingKeyIds`
Lo llama	Ambos repositorios, `RecoveryService.retry`, `DataMaintenanceService`, `upgradeEnvelope`
Riesgo al modificar	**Crítico**

Acepta dos formas: el sobre actual (con `version`) y el **heredado**, que es el objeto `payload` suelto, sin envoltura. La detección es `!envelope.containsKey('version')`.

`CipherEnvelopeInfo inspect(String encoded)`

Devuelve versión, algoritmo, `keyId` y si es heredado, **sin descifrar**. Los repositorios lo usan para decidir si toca actualizar el sobre tras una lectura correcta.

`Future<String> upgradeEnvelope(String encoded)`

Devuelve el mismo texto si ya está en la versión actual y con la llave activa; en caso contrario descifra y vuelve a cifrar. Es una operación **cara**: se llama solo cuando `inspect` confirma que hace falta.

`void activateKey(String keyId)` · `Future<void> forgetKey(String keyId)`

`activateKey` lanza `ArgumentError` con un identificador vacío. `forgetKey` borra la llave del almacén **y** la quita de `_missingKeyIds`; solo lo llama el manejador de fallo de la rotación.

Rotación — `DataMaintenanceService.rotateEncryptionKey()`

```
Future<EncryptionRotationResult> rotateEncryptionKey()
```

Aspecto	Detalle
Retorno	`EncryptionRotationResult(keyId, historyRecords, inventorySessions)`
Excepciones	`StateError('encryption_rotation_in_progress')` si ya hay una en curso; `StateError('history_payload_missing:<id>')` o `StateError('inventory_payload_missing:<id>')` si un registro no tiene `payload`; cualquier error de cifrado
Efectos secundarios	Crea una llave `v3_<epoch_ms>`; reescribe todos los `payload`; actualiza `encryption_active_key_id`; cambia la llave activa del cifrador en memoria
Lo llama	`SettingsScreen._rotateEncryptionKey`
Riesgo al modificar	**Crítico**

Secuencia exacta:

1. bandera `_rotationInProgress` para impedir reentrada;
2. leer todos los registros de ambos almacenes;
3. **precalcular** todos los sobres nuevos en memoria (descifrar con la llave vieja, cifrar con la nueva);
4. una sola transacción que escribe registros **y** metadatos;
5. `_cipher.activateKey(keyId)` solo después de que la transacción confirme;
6. si algo falla, `forgetKey(keyId)` dentro de un `try` anidado que no oculta el error original, y `rethrow`.

El paso 3 antes del 4 es deliberado: mantiene la base bloqueada el mínimo tiempo. El paso 6 es lo que impide dejar material criptográfico huérfano que después pudiera confundirse con una llave activa. Ambos tienen prueba dedicada en `test/core/data_maintenance_test.dart`.

Análisis de riesgo — `ScanSecurityAnalyzer`

`static SecurityAssessment analyze(String rawValue)`

Aspecto	Detalle
Retorno	`SecurityAssessment(level, reasons, canOpen, normalizedHost)`
Excepciones	Ninguna. Todas las rutas devuelven un valor
Efectos secundarios	Ninguno: función pura
Lo llama	`ScanRecord.fromBarcode`, `ScanRecord.manual`
Riesgo al modificar	**Alto**: es la única defensa antes de una acción externa

Orden de evaluación —importa, porque las primeras ramas devuelven de inmediato:

#	Condición	Resultado
1	Empieza por `otpauth:`, `bitcoin:`, `lightning:` o `ethereum:`	`caution`, `canOpen: false`, un motivo
2	Empieza por `mailto:`, `tel:`, `sms:`, `smsto:` o `geo:`	`low`, `canOpen: **true**`, sin motivos
3	Contenido estructurado conocido (`wifi:`, `begin:vcard`, `mecard:`, `matmsg:`, `begin:vevent`, `binary-base64:`, `000201`, `spc\n`, `bcd\n`)	`low`, `canOpen: false`
4	No es una URI `http`/`https` **y** empieza por un esquema	`high`, `canOpen: false`, «esquema de aplicación no permitido»
5	No es una URI `http`/`https` y no tiene esquema	`low`, `canOpen: false`
6	Es `http`/`https`	Se evalúan las 16 señales

Las 16 señales, en el orden en que se aplican:

#	Señal	Nivel
1	Esquema `http` sin TLS	`caution`
2	Host vacío	`high`
3	`userInfo` presente (credenciales o `@` engañoso)	`high`
4	Host contiene `xn--` (Punycode)	`high`
5	Host mezcla latino, cirílico o griego	`high`
6	Host con puntos de código > 127	`caution`
7	Host es una IP (IPv4 o contiene `:`)	`caution`
8	Host privado, `localhost` o `.local`	`high`
9	Host en la lista de 11 acortadores conocidos	`caution`
10	Más de 5 etiquetas en el host	`caution`
11	Puerto explícito distinto de 80 y 443	`caution`
12	Longitud total > 240 caracteres	`caution`
13	Contiene `%00` o un punto de código < 32	`high`
14	La ruta termina en una de 17 extensiones ejecutables o comprimidas	`high`
15	Ruta o consulta contienen `login`, `signin`, `verify`, `account`, `password`, `wallet` o `bank`	`caution`

16	Tres o más parámetros de seguimiento (`utm_*`, `gclid`, `fbclid`, `mc_cid`, `ref`)	`caution`

`caution` solo eleva desde `low`; `high` es absorbente. `canOpen` para una URI web es siempre `true` **independientemente del nivel**: un enlace de riesgo alto sigue siendo abrible, pero el diálogo de confirmación es obligatorio porque `_openRecord` fuerza `shouldConfirm` cuando `riskLevel != RiskLevel.low`.

Las 17 extensiones peligrosas: `.apk .appx .bat .cmd .com .dmg .exe .hta .iso .jar .js .msi .pkg .ps1 .scr .vbs .zip`.

Los 11 acortadores: `bit.ly tinyurl.com t.co goo.gl ow.ly buff.ly is.gd cutt.ly rebrand.ly shorturl.at tiny.one`.

`static Uri? normalizedActionUri(String rawValue)`

Devuelve la URI que debe abrirse. Para los esquemas de sistema devuelve `Uri.tryParse` tal cual; para el resto aplica `_toWebUri`, que **antepone `https://` si el valor empieza por `www.`**. Es la única normalización que hace el analizador.

Interpretación — `ContentInterpreter.parse(String rawValue)`

Aspecto	Detalle
Retorno	`ParsedContent(kind, title, fields, summary, sensitive)`
Excepciones	Ninguna: todas las ramas devuelven, y la última es `ContentKind.text`
Efectos secundarios	Ninguno
Lo llama	`ContentParserRegistry.parse` y `LegacyContentParser.parse`
Riesgo al modificar	Medio. `sensitive` decide qué **no** se persiste

Orden de reconocimiento (el primero que coincide gana):

#	Prefijo o patrón	`ContentKind`	`sensitive`
1	`binary-base64:`	`binary`	no
2	`wifi:`	`wifi`	**sí si `P` no está vacío**
3	`begin:vcard`	`contact`	no
4	`mecard:`	`contact`	no
5	`begin:vevent`	`event`	no
6	`mailto:` o `matmsg:`	`email`	no
7	`tel:`	`phone`	no
8	`sms:` o `smsto:`	`sms`	no
9	`geo:`	`geo`	no

10	`otpauth:`	`otp`	**sí, siempre**
11	`bitcoin:`, `lightning:`, `ethereum:`	`crypto`	**sí, siempre**
12	Empieza por `SPC\n`	`payment` (Swiss QR Bill)	**sí**
13	Empieza por `BCD\n`	`payment` (EPC/SEPA)	**sí**
14	Empieza por `000201` y contiene `6304`	`payment` (EMVCo)	**sí**
15	Empieza por `@` y contiene `ANSI`	`identity` (AAMVA)	**sí**
16	Empieza por `(NN)` o contiene `<GS>`	`gs1`	no
17	10 dígitos, o 13 empezando por `978`/`979`	`isbn`	no
18	URI `http`/`https` con host, o empieza por `www.`	`url`	no
19	8 a 14 dígitos	`product`	no
20	Cualquier otra cosa	`text`	no

Cuidado con el orden 17 antes que 19: un EAN-13 que empiece por `978` o `979` se clasifica como ISBN, no como producto. Es correcto —los prefijos 978 y 979 son precisamente el rango Bookland de ISBN— pero explica por qué un libro escaneado aparece como «ISBN» y no como «Código de producto».

Y **cuidado con la comprobación `value.length == 10`** de `_isIsbn`: cualquier cadena de exactamente 10 caracteres tras eliminar todo lo que no sea dígito o `X`/`x` se considera ISBN. Un código de producto de 10 dígitos se etiquetará como ISBN. R-14.

Funciones auxiliares relevantes

Función	Qué hace	Nota
`_segments(String)`	Divide por `;` respetando `\` como escape y separa clave/valor por el primer `:`	Es lo que permite interpretar `WIFI:S:Mi\;Red;P:clave\:segura;;`. Hay prueba dedicada
`_tlv(String)`	Recorre un TLV EMVCo: 2 caracteres de identificador, 2 de longitud, N de valor	Se detiene en el primer campo mal formado en vez de lanzar
`_aiLabel(String)`	Traduce 9 Application Identifiers GS1 a nombre legible	El resto se muestra como `AI NN`
`_aamvaLabel(String)`	Traduce 7 códigos AAMVA	Solo esos 7 se extraen

Registro de parsers — `ContentParserRegistry`

Método	Firma	Nota

`parse`	`ParsedContent parse(String rawValue)`	Recorre por prioridad descendente; devuelve el primero que acepte
`register`	`void register(ContentParser parser)`	Sustituye por `id` y reordena
`unregister`	`void unregister(String id)`	**No puede quitar `builtin-v2`**
`parsers`	`List<ContentParser> get`	Copia inmodificable

`ContentParser` exige `id`, `priority`, `canParse` y `parse`. `LegacyContentParser` tiene prioridad `-1000` para quedar siempre al final.

Historial — `HistoryRepository`

Método	Firma	Efectos	Riesgo
`initialize`	`Future<HistoryMigrationStatus> initialize()`	Ejecuta la migración desde preferencias	**Alto**
`load`	`Future<List<ScanRecord>> load({int limit = maxItems})`	Descifra; **actualiza sobres antiguos**; registra incidencias	Medio
`upsert`	`Future<void> upsert(ScanRecord)`	Escribe y llama a `_trim()`	Bajo
`upsertAll`	`Future<void> upsertAll(Iterable<ScanRecord>)`	Precifra, escribe en una transacción, `_trim()`	Medio
`replaceAll`	`Future<void> replaceAll(Iterable<ScanRecord>)`	Borra todo y escribe en una transacción. **No llama a `_trim()`**	**Alto**
`ids`	`Future<Set<String>> ids()`	Solo claves; no descifra	Bajo
`remove`	`Future<void> remove(String id)`	Borra un registro	Bajo
`pruneOlderThan`	`Future<void> pruneOlderThan(int days)`	No hace nada si `days <= 0`	Medio
`clear`	`Future<void> clear()`	Vacía el almacén	Alto

`load` tiene un efecto secundario que su nombre no anuncia: si el sobre es heredado o de versión anterior, lo reescribe con `upgradeEnvelope`. Es actualización perezosa deliberada, documentada en `../quality/MIGRATIONS.md`, pero significa que **leer el historial escribe en la base**.

Y otro: si un registro no descifra, `load` **no lanza**. Lo omite del resultado y crea una incidencia con `code: 'decrypt_<TipoDeError>'` y la carga cifrada guardada. Un registro dañado no impide cargar los demás.

`replaceAll` no llama a `_trim()`. En la práctica no puede desbordar, porque `ImportService.maxHistoryRecords` vale también `5000`, pero la protección depende de esa coincidencia entre dos constantes de archivos distintos. R-15.

`HistoryMigrationStatus`

```
const HistoryMigrationStatus({
 required bool completed,
 int migrated = 0,
 String? errorCode,
 bool backupPresent = false,
})
```

`errorCode` tiene la forma `legacy_<TipoDeError>`; es lo que se muestra como subtítulo en Ajustes cuando la migración quedó pendiente.

Importación — `ImportService`

Método	Firma	Excepciones
`pickHistoryJson`	`Future<HistoryImportPreview?> pickHistoryJson({required Set<String> existingIds})`	Las de `parseHistoryBytes`, más `FormatException` por tamaño en el propio selector
`parseHistoryBytes`	`HistoryImportPreview parseHistoryBytes(List<int> bytes, {required Set<String> existingIds, String fileName})`	`FormatException` con mensaje en español para: >25 MB, JSON demasiado profundo, demasiados nodos, procedencia incorrecta, esquema futuro, sin registros, >5 000 registros
`pickInventoryJson`	`Future<InventoryImportPreview?> pickInventoryJson()`	Idem
`parseInventoryBytes`	`InventoryImportPreview parseInventoryBytes(List<int> bytes, {String fileName})`	Idem, más «La sesión supera el límite de productos»

`parseHistoryBytes` **nunca escribe**. Devuelve un `HistoryImportPreview` con `valid`, `duplicates`, `rejected`, `schemaVersion`, `legacy` y `fileName`. La escritura la decide `ScanStore.importPreview` tras elegir estrategia.

Los registros individuales que no se pueden reconstruir se cuentan en `rejected` y **no abortan la importación**: solo un error de forma del archivo completo lanza.

`_validateJsonShape` recorre el árbol con una pila explícita, no por recursión. Es la defensa que evita un desbordamiento de pila con un JSON profundo; el límite de profundidad se comprueba **por rama**, y el de nodos, en total.

Exportación — `ExportService`

Método	Formato	Nombre de archivo	MIME
`shareHistoryJson`	Sobre con `application`, `schemaVersion: 2`, `exportedAt`, `type: history`, `records`	`historial-escaneos.json`	`application/json`
`shareHistoryCsv`	8 columnas	`historial-escaneos.csv`	`text/csv`
`shareHistoryXlsx`	9 columnas (añade `Notas`)	`historial-escaneos.xlsx`	`…spreadsheetml.sheet`
`shareInventoryJson`	Sobre con `type: inventory` y `session`	`inventario-<id>.json`	`application/json`
`shareInventoryCsv`	8 columnas	`inventario-<id>.csv`	`text/csv`

`shareInventoryXlsx`	8 columnas	`inventario-<id>.xlsx`	`…spreadsheetml.sheet`

Asimetría real: el CSV del historial tiene 8 columnas y **no incluye `notas`**; el XLSX tiene 9 y sí. Los dos del inventario tienen ambos 8 y los dos incluyen `notas`. R-16.

Todo lo exportado está **en claro**. Es una decisión explícita del usuario y está declarada en `docs/SECURITY.md`.

Confirmación de lectura — `ScanFeedback`

Miembro	Firma	Comportamiento
`success`	`Future<void> success({required bool sound, required bool vibration})`	Vibra y suena, cada canal según su ajuste. **Ningún fallo se propaga**
`dispose`	`Future<void> dispose()`	Libera el reproductor; tolera que nunca se haya creado
`toneUnavailable`	`bool get` (`@visibleForTesting`)	`true` tras el primer fallo del reproductor

`AssetScanTonePlayer` construye el `AudioPlayer` **en la primera reproducción**, no en el constructor: una pantalla que nunca lee un código no crea un reproductor de plataforma. Configura `mixWithOthers` (no interrumpe música ni llamadas), `PlayerMode.lowLatency`, `ReleaseMode.stop` y precarga la fuente. Antes de cada reproducción hace `seek(Duration.zero)`: sin eso, una segunda lectura llegada mientras el tono suena sería silenciosa.

Modelos — firmas clave

`ScanRecord`

Factoría	Firma	Nota
`fromBarcode`	`ScanRecord.fromBarcode(Barcode barcode, {required String source, DateTime? scannedAt})`	Ejecuta analizador e intérprete
`manual`	`ScanRecord.manual({required String rawValue, required String format, required String source})`	Igual, pero sin `Barcode`
`fromJson`	`ScanRecord.fromJson(Map<String, dynamic> json)`	**Reinterpreta** si falta `parsed` o `contentType`

Método estático	Firma	Nota
`payloadForBarcode`	`static String payloadForBarcode(Barcode)`	Devuelve el texto, o `binary-base64:<base64>`, o cadena vacía
`_decodedBytes`	`static Uint8List? _decodedBytes(BarcodeBytes?)`	Distingue `DecodedBarcodeBytes` de `DecodedVisionBarcodeBytes` con comprobación de tipo explícita

`isSensitive` delega en `parsed.sensitive`. Es el único predicado que decide qué queda fuera del historial automático.

`InventorySession` · `InventoryItem`

`InventorySession.items` está indexado por el **valor bruto del código**, no por un identificador propio. `totalUnits` suma las cantidades. `isOpen` es `closedAt == null`.

`InventoryItem.copyWith` acepta solo `quantity`, `lastScannedAt` y `notes`: el código, el formato, la etiqueta y la primera lectura son inmutables una vez creados.

Claves de `SharedPreferences`

Escritas por `SettingsRepository.save` (17):

`theme_mode`, `language`, `sound_enabled`, `vibration_enabled`, `save_history`, `private_mode`, `auto_torch`, `use_scan_window`, `confirm_before_open`, `hide_sensitive_values`, `biometric_lock`, `high_contrast`, `large_controls`, `reduce_motion`, `clear_clipboard_seconds`, `history_retention_days`, `feature_flags`.

Escritas por `HistoryRepository` (4): `scan_history_v1`, `scan_history_v1_backup`, `history_migrated_to_v2`, `history_migration_error`.

Leída y eliminada por `EncryptionMetadataRepository` (1, solo migración): `encryption_active_key_id`.

`resetNonSensitive()` borra 14 de las 17 primeras. **Deja intactas `save_history`, `private_mode` y `biometric_lock`**: restablecer las preferencias visuales desde la pantalla de inicio seguro no puede desactivar el bloqueo de la aplicación ni reactivar el guardado de historial que el usuario había apagado. Es una decisión deliberada y **no documentada en el repositorio** fuera del propio nombre del método.

Almacenes Sembast

Almacén	Tipo	Clave	Valor
`scan_history`	`stringMapStoreFactory`	`ScanRecord.id`	`{id, scannedAt, payload}`
`inventory_sessions`	`stringMapStoreFactory`	`InventorySession.id`	`{id, createdAt, payload}`
`recovery_issues`	`stringMapStoreFactory`	SHA-256 truncado a 24	`RecoveryIssue.toJson()`
`_schema_meta`	`StoreRef<String, Object?>`	Nombre de la marca	Valor escalar
`_security_meta`	`StoreRef<String, Object?>`	`encryption_active_key_id`	Cadena

Detalle en 07-database.md.

Códigos de error e identificadores internos

Código	Dónde se genera	Significado
`encryption_key_missing:<keyId>`	`PayloadCipher`	La llave no está en el almacén
`encryption_rotation_in_progress`	`DataMaintenanceService`	Rotación reentrante
`history_payload_missing:<id>`	idem	Un registro de historial sin `payload`
`inventory_payload_missing:<id>`	idem	Un registro de inventario sin `payload`
`migration_limit_exceeded`	`HistoryRepository`	La migración superaría 5000 registros

`migration_verification_failed`	idem	La transacción no contiene todo lo esperado
`missing_payload`	Ambos repositorios	Incidencia de recuperación
`decrypt_<TipoDeError>`	Ambos repositorios	Incidencia de recuperación
`legacy_<TipoDeError>`	`HistoryRepository`	Fallo de la migración desde preferencias
`legacy_not_list`	idem	El historial heredado no era una lista JSON

Las áreas que se pasan a `AppDiagnostics.record` son cinco: `flutter`, `platform`, `root_zone`, `bootstrap` y `bootstrap_temporary`. Las tres primeras las instala `installGlobalErrorHandlers()`; las dos últimas las emite `_BootstrapHostState._retry` según el modo con el que se reintentó el arranque.

Límites operativos

Límite	Valor	Dónde
Imágenes por lote de galería	20	`pickMultiImage(limit: 20)`
Páginas de PDF por lote	50	`PdfPageRenderer.pickAndRender(maxPages: 50)`
Escala de renderizado de página	2400 px en el lado mayor, acotada a `[1.5, 4.0]`	`_renderScale`
Registros de historial	5000	`HistoryRepository.maxItems`
Ventana antirrebote de la cámara	2 s por firma de captura	`ScannerScreen._handleCapture`
Ventana antirrebote del inventario	1200 ms por código	`InventoryScreen._onDetect`
Ancho máximo de la interfaz	560 px lógicos	`HandheldFrame.maxWidth`
Lado del marco de lectura	68 % del lado menor	`ScannerScreen.build`
Duración del barrido del marco	2200 ms, con inversión	`_ScannerOverlayState`
Entradas de diagnóstico	100	`AppDiagnostics.maxEntries`

Continúa en 06 · Explicación profunda del código.